Lesson 1:
Introduction and Agents

Shreya Shankar & Hamel Husain

Instructor backgrounds



Shreya Shankar

- CS professor at CMU
- 10+ papers in MLOps and LLMOps
- Helped many teams build AI products

sh-reya.com



Hamel Husain

- 25 years ML experience: Airbnb, GitHub
- Open source work on MLOps
- Independent consultant, 30+ companies

hamel.dev

Teaching assistants



Antaripa Saha

- Applied AI Consultant at Parlance Labs
- LLMs, agents, search, and evaluations
- Previously led NLP at SproutsAI

antaripasaha.com



Eleanor Berger

- Member of Technical Staff at Jimini Health
- Former principal engineering leader at Microsoft and Google
- Helps engineers and teams use agentic AI through Agentic Ventures

intellectronica.net

Who is in the room

400+

companies across previous cohorts

Adobe, Airbnb, Amazon, Capital One, Cisco, Coinbase, Duolingo, Ford, Google, Hugging Face, IBM, JP Morgan, Meta, Microsoft, Netflix, NVIDIA, OpenAI, Salesforce, Stanford, Walmart, and many more

- Engineers, product managers, data scientists
- Building real AI products at work
- Talk to each other in Discord

What you want to learn

- "I need to implement a strategy to improve the agent. We have offline evals too, but not really a concrete strategy." - Shekhar
- "Leave with a method I can apply to a capability that doesn't exist yet in our org." - Corey
- "Automate and scale a very manual process I currently use for evals." - Alice
- "Deepen that foundation so I can define evaluation strategy and use evals to guide product decisions." - Aastha

Course structure

Lectures

9 sessions across 5 modules

Office hours

10+ sessions, open Q&A

Homework

10 coding assignments, video deliverables

Course reader

150+ pages, released by module

TAs and Discord

Staff across time zones, 1,000+ student community

Guest lectures

Industry practitioners on real eval systems

Course modules

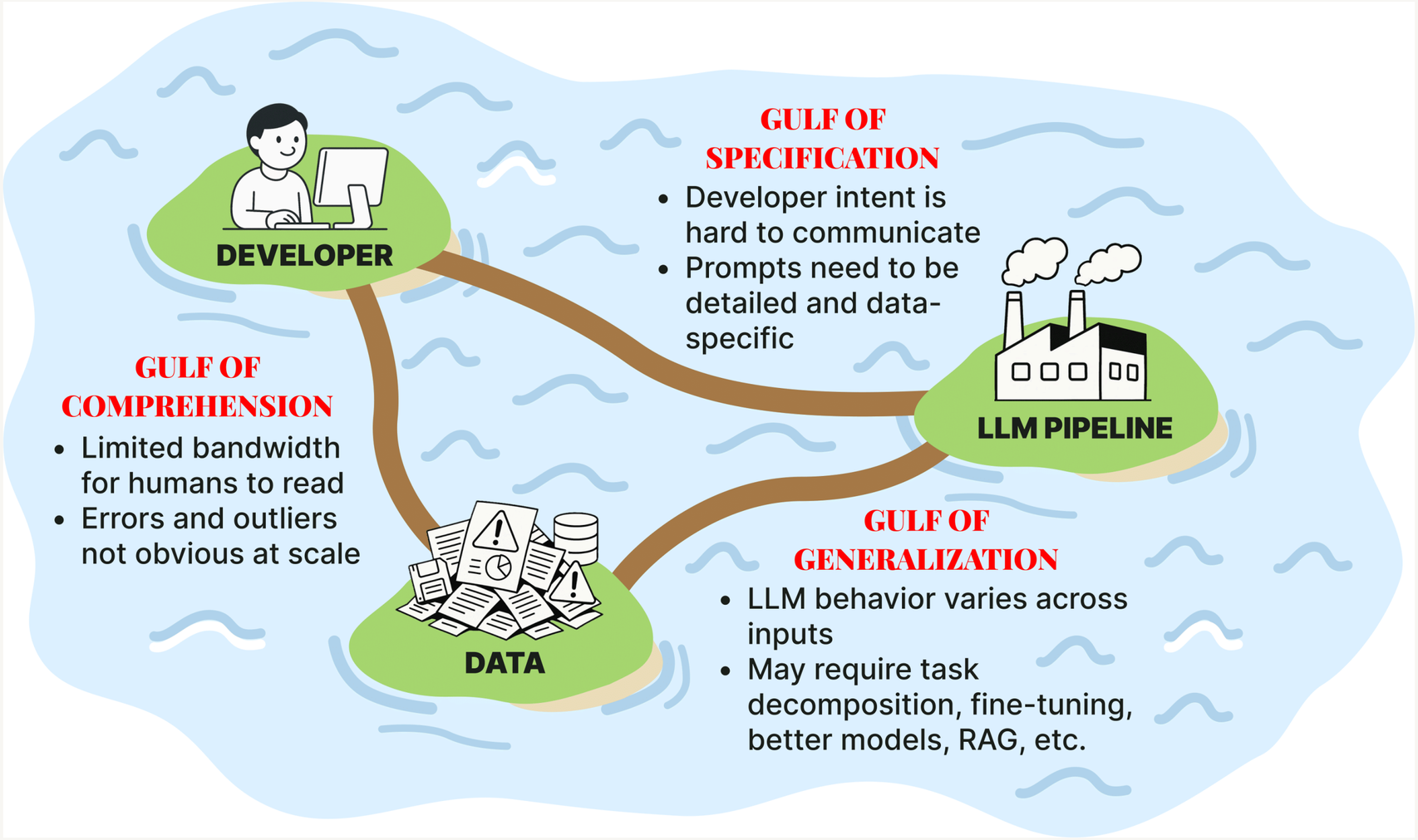
Module	What you learn	Lectures	HW
1. Build	Build and instrument an agent for observability OpenAI Agents SDK · OpenTelemetry · Langfuse	3	3
2. Error analysis	Find and categorize failures LLM judges · Langfuse	2	2
3. CI/CD	Automate checks, monitor in production GitHub Actions · Langfuse	1	2
4. Security	Red-team and defend the agent Promptfoo	1	1
5. Improve	Improve accuracy, reduce cost GEPA · Prompt Caching · Model Cascades	2	2

Today

- Three gulfs: why it's hard to build agents
- Building an e-commerce agent (running course example)
- Different ways to assemble agents
- Homework 1 walkthrough

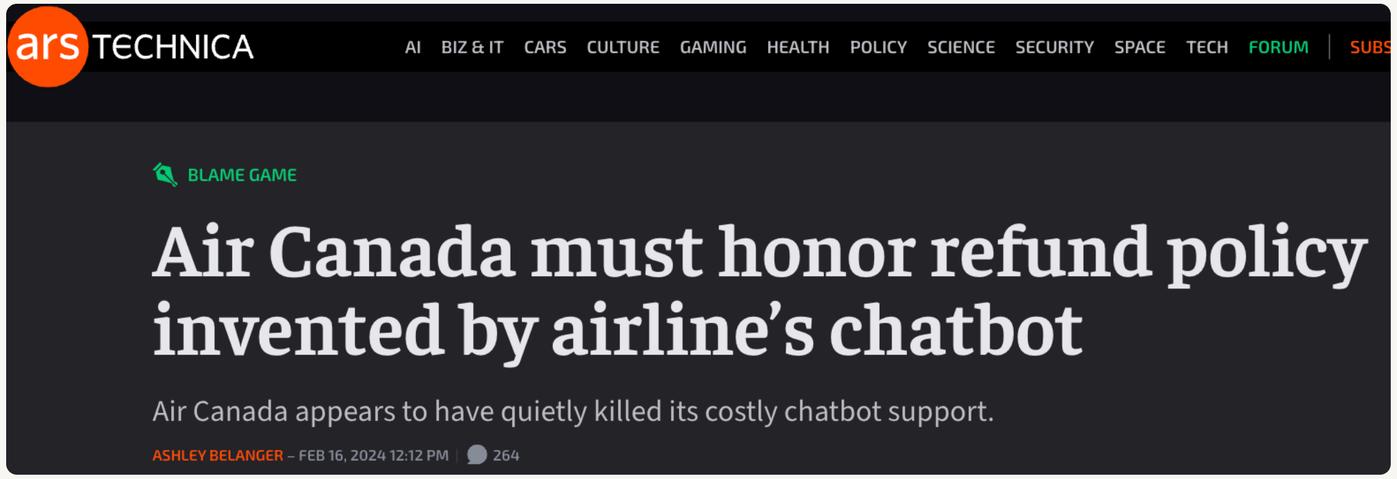
The three gulfs

Three gulfs of LLM agent development



Adapted from DocWrangler, Shankar et al. 2025

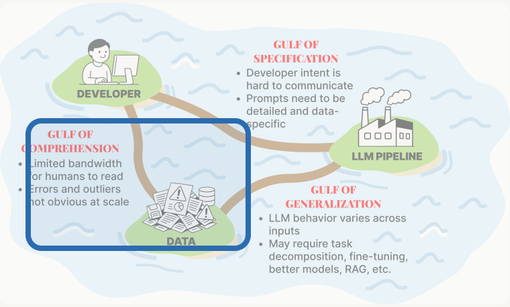
Example application



The chatbot invented a refund policy. A tribunal ruled Air Canada liable.

We classify the challenges into three gulfs.

Gulf of Comprehension

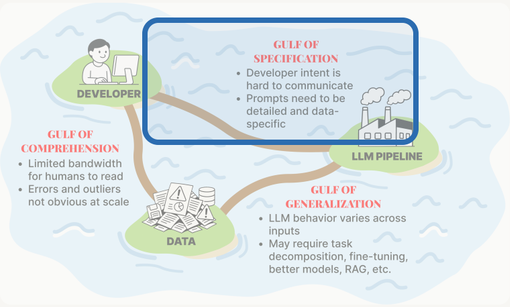


You need to read your inputs and outputs. Inputs to spot adversarial or outlier queries. Outputs to catch where the model is wrong.

Air Canada: nobody read the outputs. The chatbot was confidently describing a nonexistent policy, and no one caught it until a customer sued!

We solve it with systematic error analysis: reviewing traces at scale instead of spot-checking.

Gulf of Specification

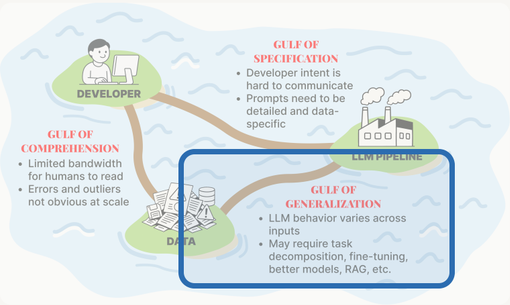


Translating what you want into instructions the model follows is hard. Gaps in the spec become gaps in behavior.

Air Canada: nobody specified which policies exist and which don't. The model filled the gaps with plausible fiction.

We solve it with good prompt hygiene and discipline: know exactly what's in your prompt, what your tools do, and where the gaps are.

Gulf of Generalization



Even with a good spec, the model doesn't apply it consistently. Works on common queries, breaks on the long tail.

Air Canada: standard questions worked fine. Bereavement fares were rare enough that nobody tested them.

We solve it by building tests from real failures, running on every change, and improving (prompts, models, tools) grounded in what the tests show.

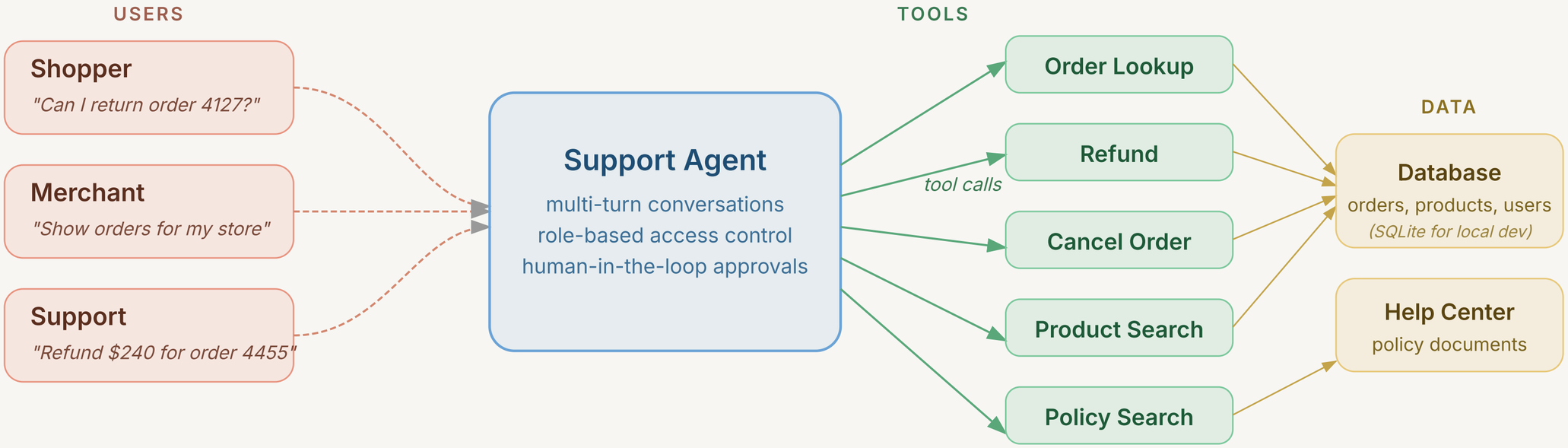
Building Cartwheel

The running example

Cartwheel is a fictional e-commerce platform. You build its customer support agent and evaluate it across all five modules.

- Shoppers ask about orders, returns, refunds
- Merchants manage their stores
- Support staff can access any order
- The agent retrieves policies, calls tools, and makes decisions

The Cartwheel stack



Writing a good spec

Before you write a prompt, write down what the agent should and should not do. Human-written, structured, as concrete as possible.

- **Scope:** what requests does the agent handle? What does it refuse?
- **Permissions:** who can see what data? Who can change what?
- **Tool contracts:** what does each tool return on success and failure?
- **Escalation:** when does the agent hand off to a human?

> The spec is not the prompt, but every rule in the spec should be fully covered by the prompt.

Spec: Scope

Request type	Agent handles?
Order status, delivery tracking	Yes
Return and refund requests	Yes
Product search, policy questions	Yes
Account changes (email, password)	Refuse, direct to settings
Payment method updates	Refuse, direct to settings
Complaints about other customers	Refuse, escalate

Spec: Permissions

The three platform roles ↓

Capability	Shopper	Merchant	Support
View own orders	Yes		
View store orders		Own store	Any store
Search products / policies	Yes	Yes	Yes
Issue refund under $100			Yes
Issue refund over $100			Queued
Cancel order	Own, pre-ship		Yes

Spec: Tool contracts

Tool	On success	On failure
`get_order`	Order details	`permission_denied`
`issue_refund`	Refund confirmation	`not_eligible` or `queued_for_approval`
`cancel_order`	Cancellation confirmed	`already_shipped`
`search_products`	Product list	Empty results
`get_policy`	Policy text	Not found

Spec: Escalation

If the agent gets it wrong, what's the cost? That determines who decides.

Risk tier	Examples	Who decides
Read-only	Order lookup, product search, policy questions	Agent
Low-risk write	Cancel pre-shipment order, small refund, apply promo code	Agent
High-risk write	Large refund, post-shipment cancel, change email, update payment	Human only

The spec and the system prompt

Structured spec

Scope
- Help with orders and refunds.
- Answer product and policy questions.

Permissions
- Shoppers can view only their own orders.

Tool contracts
- Check order eligibility before issuing a refund.

Escalation
- Refunds above $100 need human approval.

... remaining spec ...

Example system prompt

You are Cartwheel's support assistant. Help with order status, refunds, product and policy questions.

Use the authenticated session to identify the user. Shoppers may access only their own orders.

Before issuing a refund, look up the order and check its refund eligibility. Refunds above $100 require human approval.

If a tool returns permission_denied, explain the restriction. Do not try to bypass it.

... continued with tool guidance, refusal rules, and response style ...

Real world system prompts

Codex

```
You are Codex, based on GPT-5. You are running as a coding agent in the Codex CLI on a user's computer.

## General

- When searching for text or files, prefer using `rg` or `rg --files` respectively because `rg` is much faster than alternatives like `grep`. (If the `rg` command is not found, then use alternatives.)

## Editing constraints

- Default to ASCII when editing or creating files. Only introduce non-ASCII or other Unicode characters when there is a clear justification and the file already uses them.
- Add succinct code comments that explain what is going on if code is not self-explanatory. You should not add comments like "Assigns the value to the variable", but a brief comment might be useful ahead of a complex code block that the user would otherwise have to spend time parsing out. Usage of these comments should be rare.
- Try to use apply_patch for single file edits, but it is fine to explore other options to make the edit if it does not work well. Do not use apply_patch for changes that are auto-generated (i.e. generating package.json or running a lint or format command like gofmt) or when scripting is more efficient (such as search and replacing a string across a codebase).
- You may be in a dirty git worktree.
    * NEVER revert existing changes you did not make unless explicitly requested, since these changes were made by the user.
    * If asked to make a commit or code edits and there are unrelated changes to your work or changes that you didn't make in those files, don't revert those changes.
    * If the changes are in files you've touched recently, you should read carefully and understand how you can work with the changes rather than reverting them.
    * If the changes are in unrelated files, just ignore them and don't revert them.
- Do not amend a commit unless explicitly requested to do so.
- While you are working, you might notice unexpected changes that you didn't make. If this happens, STOP IMMEDIATELY and ask the user how they would like to proceed.
- **NEVER** use destructive commands like `git reset --hard` or `git checkout --` unless specifically requested or approved by the user.

## Plan tool

When using the planning tool:
- Skip using the planning tool for straightforward tasks (roughly the easiest 25%).
- Do not make single-step plans.
- When you made a plan, update it after having performed one of the sub-tasks that you shared on the plan.

## Special user requests
```

[OpenAI's public GPT-5 Codex prompt]

Claude Code

```
You are a Claude agent, built on Anthropic's Claude Agent SDK.

---


You are an interactive CLI tool that helps users with software engineering tasks. Use the instructions below and the tools available to you to assist the user.

IMPORTANT: Assist with authorized security testing, defensive security, CTF challenges, and educational contexts. Refuse requests for destructive techniques, DoS attacks, mass targeting, supply chain compromise, or detection evasion for malicious purposes. Dual-use security tools (C2 frameworks, credential testing, exploit development) require clear authorization context: pentesting engagements, CTF competitions, security research, or defensive use cases.
IMPORTANT: You must NEVER generate or guess URLs for the user unless you are confident that the URLs are for helping the user with programming. You may use URLs provided by the user in their messages or local files.

If the user asks for help or wants to give feedback inform them of the following:
- /help: Get help with using Claude Code
- To give feedback, users should report the issue at https://github.com/anthropics/claude-code/issues

# Tone and style
- Only use emojis if the user explicitly requests it. Avoid using emojis in all communication unless asked.
- Your output will be displayed on a command line interface. Your responses should be short and concise. You can use Github-flavored markdown for formatting, and will be rendered in a monospace font using the CommonMark specification.
- Output text to communicate with the user; all text you output outside of tool use is displayed to the user. Only use tools to complete tasks. Never use tools like Bash or code comments as means to communicate with the user during the session.
- NEVER create files unless they're absolutely necessary for achieving your goal. ALWAYS prefer editing an existing file to creating a new one. This includes markdown files.
- Do not use a colon before tool calls. Your tool calls may not be shown directly in the output, so text like "Let me read the file:" followed by a read tool call should just be "Let me read the file." with a period.

# Professional objectivity
Prioritize technical accuracy and truthfulness over validating the user's beliefs. Focus on facts and problem-solving, providing direct, objective technical info without any unnecessary superlatives, praise, or emotional validation. It is best for the user if Claude honestly applies the same rigorous standards to all ideas and disagrees when necessary, even if it may not be what the user wants to hear. Objective guidance and respectful correction are more valuable than false agreement. Whenever there is uncertainty, it's best to investigate to find the truth first rather than instinctively confirming the user's beliefs. Avoid using over-the-top validation or excessive praise when responding to users such as "You're absolutely right" or similar phrases.
```

[Public snapshot, January 2026]

Agent architectures

Three architectures

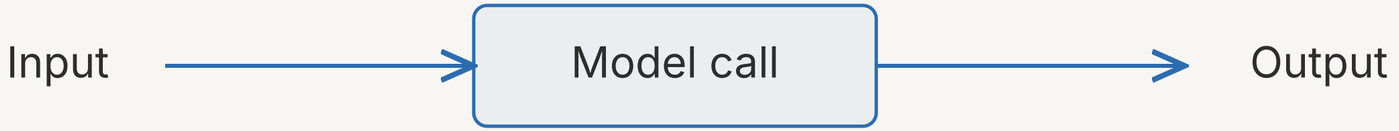
Pick the simplest one that works for your use case.

Architecture	How it works	Cost	Latency
Single call	One input, one output	Lowest	Lowest
Fixed workflow	Predefined steps, always the same	Predictable	Predictable
Agent loop	Model picks tools, iterates	Variable	Variable

Single call

One prompt in, one answer out. No tools, no state, no loop.

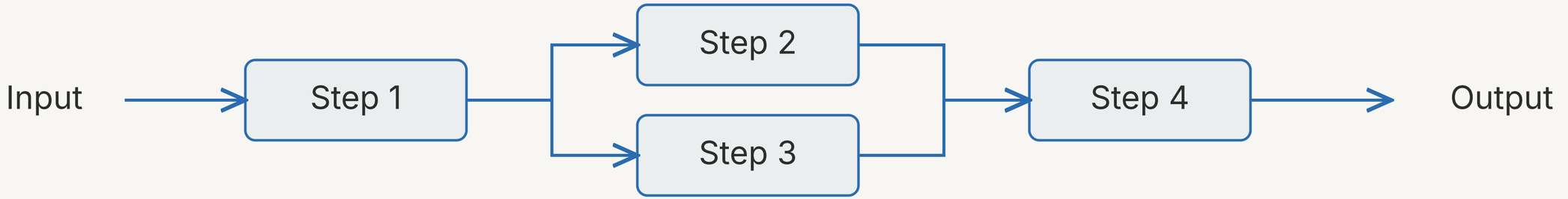
- **Example:** Gmail classifying an email as Primary, Social, or Promotions
- **Example:** Extracting structured fields from a receipt or invoice
- **Choose when:** you need high throughput, low cost, and the task fits in one prompt



Fixed workflow

Steps are predefined. The model runs at each step, but the order never changes.

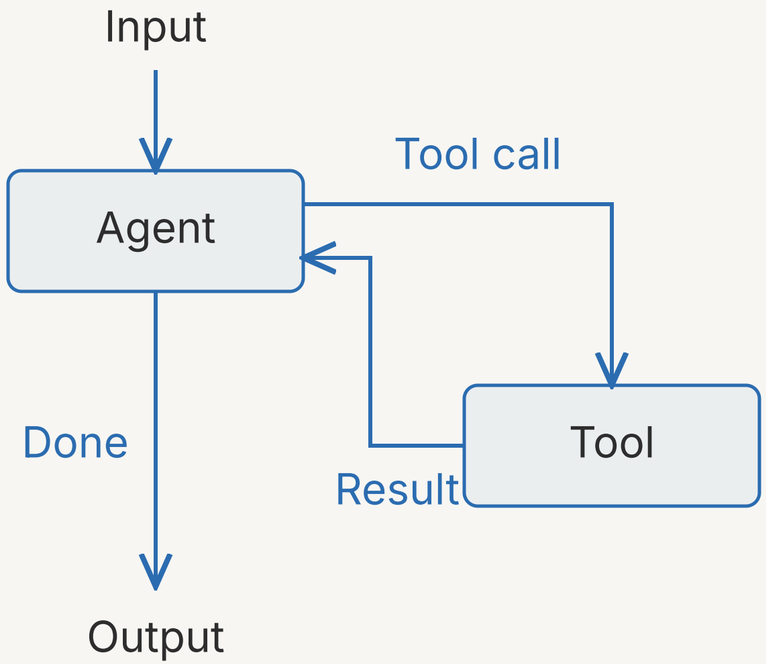
- **Example:** Perplexity: search the web, read the pages, synthesize an answer, add citations
- **Example:** Document pipeline: chunk pages, extract claims, validate against a schema, write to database
- **Choose when:** you need predictable cost and latency, and the steps are the same every time



Agent loop

The model decides which tool to call based on what it learned so far. Repeats until done.

- **Example:** Claude Code: reads files, runs tests, edits code, decides what to do next
- **Example:** Cartwheel: looks up order, checks policy, decides if eligible, issues refund
- **Choose when:** the next step depends on data from the previous step, and you accept variable cost and latency



Building the agent loop

OpenAI Agents SDK (open source)

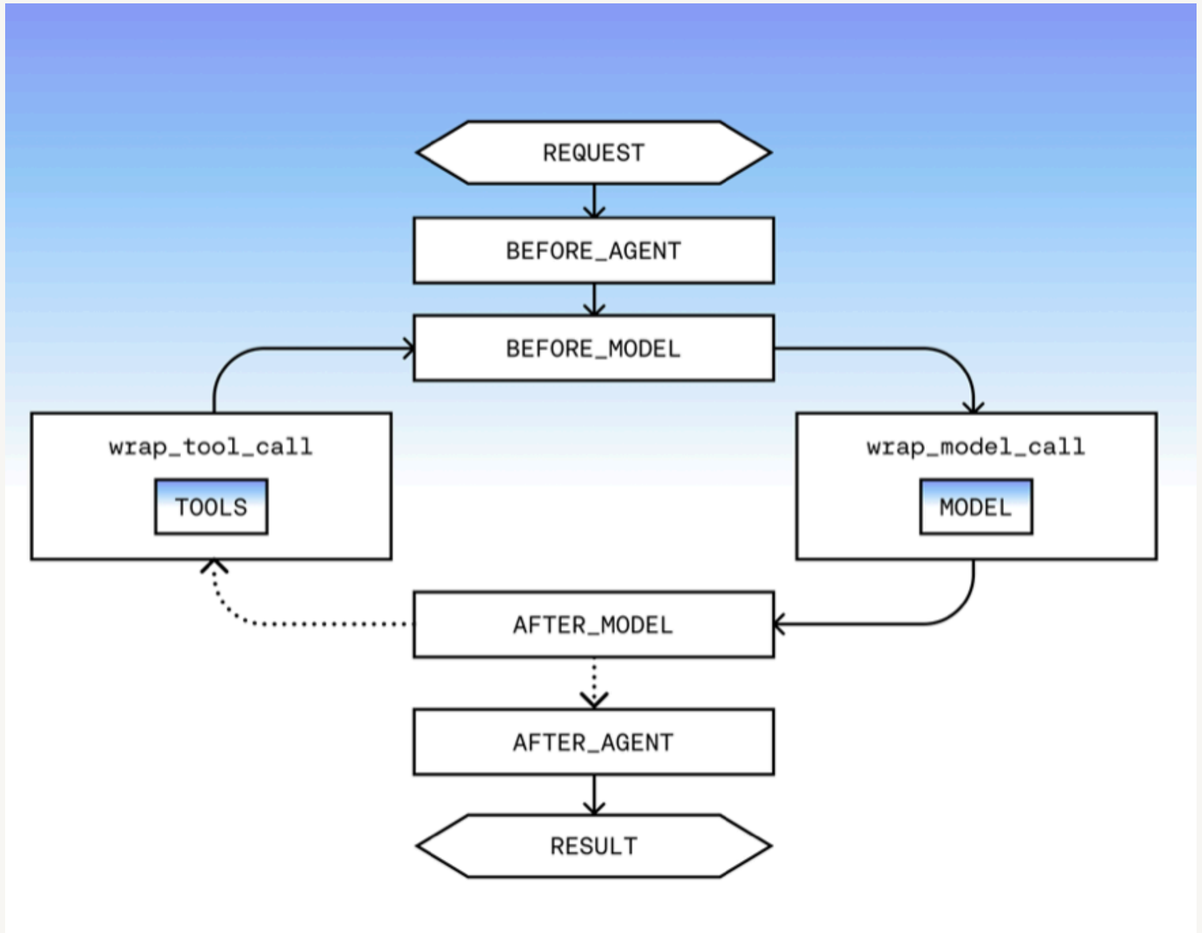
```
agent = Agent(
    model="gpt-sol",
    instructions=SYSTEM_PROMPT,
    tools=[get_order, issue_refund,
           cancel_order, get_policy],
)

result = Runner.run(
    agent,
    messages=conversation,
    context=AuthContext(role, user_id),
    max_turns=10,
)
```

- **Instructions:** system prompt with role, rules, constraints
- **Tools:** Python functions the model can call
- **Context:** who the user is (role, ID) so tools check permissions
- **Max turns:** stop the loop after N steps

The SDK also handles multi-agent routing.

Inside the agent loop



- **Model:** choose a tool or answer.
- **Harness:** run the tool, add its result, repeat.
- **Hooks:** add checks or logging around calls.
- **Stop:** final answer, limit, or error.

LangChain hook names shown. [How to Build a Custom Agent Harness]

Writing good tool descriptions

The SDK reads your **function name**, **docstring**, and **type hints** to build the tool schema the model sees.

- **Function name** becomes the tool name: `find_order` not `search_v2`
- **Docstring** becomes the description the model uses to decide when to call it
- **Parameter types + docstrings** become the argument schema (supports Google, Sphinx, NumPy formats)

> Every tool definition eats context tokens. Keep docstrings short and precise.

openai.github.io/openai-agents-python/tools

Example tool: order lookup

```
@function_tool
def find_order(ctx, query: str):
    """Search orders for the current
user."""
    user = ctx.context
    orders = db.search_orders(
        user_id=user.id, query=query
    )
    if not orders:
        return {"ok": False,
                "error":
"no_matching_orders"}
    return {"ok": True,
            "orders": [o.to_dict() for o
in orders]}
```

- **ctx.context** is the auth identity, e.g. `{role: "shopper", user_id: 1}`
- **Permission baked into the query:** `user_id=user.id` means user 2 can never see user 1's orders
- **Separate tools for separate patterns:** `find_order("earmuffs")` for fuzzy search, `get_order(4127)` for exact ID lookup

Homework 1

Homework 1

Goal: get the agent working and start noticing where it fails and why.

We provide: a starter repo with `SPEC.md`, 3 completed tools, the agent framework, and tests.

- **Part A:** Implement 5 required tools, then add more of your own
- **Part B:** Write 10+ test queries yourself (as shopper, merchant, support), run them, record what happened vs. what the spec says
- **Part C:** Find one missing prompt instruction, test whether adding it helps

github.com/ai-evals-course/cartwheel-homeworks

Before Lesson 2 (Wednesday 9/9)

- **HW1:** implement tools, run conversations, find failures
- **Read:** Background + Module 1 in the course reader
- **Watch:** Antaripa's walkthrough video for getting started with HW1
- There is a homework every lecture, so try to finish before the next one

When	What
Wed 9/9, 8 AM PT	Lecture 2: Observability
Wed 9/9, 9:30 AM PT	Office hours
Sat 9/12, 9 AM PT	Lecture 3: Synthetic Data and Scenarios

Please ask questions in Discord!